

Universidade Federal de Sergipe (UFS)
Departamento de Computação

Disciplina: Engenharia de Software II (2025.2)
Docente: Prof. Dr. Glauco de Figueiredo Carneiro

Atividade 3 - Etapa 1
Diagnóstico de DevOps e Auditoria de CI/CD
Projeto Objeto: *Scrapegraph-ai*

Equipe e Contribuições:

Nome	Matrícula	Contribuição na Atividade
Rafael Gomes O. Santos	202300095730	Investigação e Coleta de Evidências (Workflows e PRs).
Cauan Teixeira Machado	202300038627	Introdução ao Projeto e Relevância.
Maria Eduarda M. da Silva	202300038860	Análise do Fluxo de Contribuição (CONTRIBUTING.md).
José Gabriel R. G. de Almeida	202300095599	Workflow de Qualidade (QA e Testes).
Mateus da Silva Barreto	202300038879	Workflow de Segurança (CodeQL e Dependency Review).
Breno Silva do Nascimento	202300038968	Automação de Release.
José Victor Ribeiro de Jesus	202300038799	Ferramentas de Apoio (IA, Bots e Copilot).
Pedro Joaquim Silva Silveira	202300038897	Demonstração Prática e Conclusão.

Data de Entrega: 30 de Janeiro de 2026

1 Dados do Projeto

- **Nome do Projeto:** Scrapegraph-ai
- **Repositório Original:** <https://github.com/ScrapeGraphAI/Scrapegraph-ai>
- **Descrição:** Uma biblioteca Python de *web scraping* que utiliza Grandes Modelos de Linguagem (LLMs) e lógica de grafos para criar pipelines de extração de dados flexíveis e adaptáveis. O projeto permite converter páginas HTML complexas em dados estruturados (JSON) de forma inteligente.

2 Diagnóstico e Auditoria do Cenário Atual

Esta seção apresenta a análise aprofundada da maturidade de DevOps do projeto Scrapegraph-ai, baseada na investigação dos artefatos de configuração e no histórico de contribuições.

2.1 Panorama Tecnológico (AS-IS)

A auditoria revelou que o projeto adota o **GitHub Actions** como plataforma central de automação. Diferente de projetos legados ou puramente manuais, o Scrapegraph-ai demonstra um nível de maturidade avançado em Integração Contínua (CI) e Entrega Contínua (CD).

Identificamos a presença de uma esteira automatizada que garante:

1. **Verificação de Qualidade (Linting):** Uso de ferramentas como *Ruff*, *Black* e *Pylint* para garantir estilo e formatação.
2. **Testes Automatizados:** Execução de suítes de teste a cada *push*.
3. **Automação de Release:** O projeto utiliza *Semantic Release*, evidenciado pela criação automática de versões e changelogs por bots.

2.2 Evidências Técnicas da Maturidade

A robustez do processo é comprovada pelos seguintes achados:

- **Infraestrutura como Código (IaC):** A existência da pasta `.github/workflows/` contendo arquivos YAML modulares (`code-quality.yml`, `release.yml`, `test-suite.yml`) indica que o pipeline é versionado junto com o código-fonte.
- **Quality Gates Rígidos:** A análise do arquivo `code-quality.yml` revelou uma regra de bloqueio explícita: se a pontuação do *Pylint* for menor que 8, o pipeline falha e impede a integração do código.

- **Execução Contínua:** A análise do histórico de *Pull Requests* (PRs) mostrou que todas as contribuições passam por uma bateria de verificações (em média 7 checks) antes do *merge*.

3 Mapeamento do Fluxo de Trabalho (Original)

Com base na leitura dos arquivos de configuração, mapeamos o fluxo atual de contribuição. O diagrama abaixo ilustra o caminho do código desde o *Commit* até a integração na branch principal.

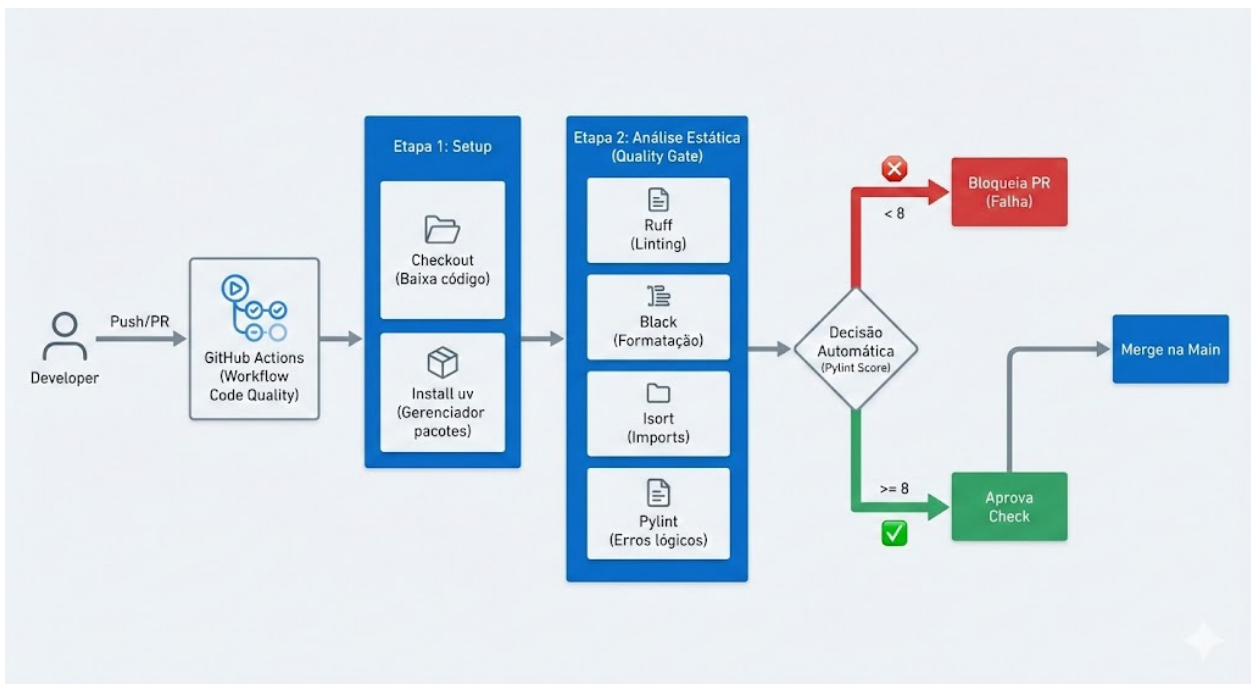


Figura 1: Fluxo de CI/CD Mapeado (AS-IS): O diagrama destaca o Quality Gate com Pylint.

O fluxo destaca-se pela etapa de "**Quality Gate**", onde ferramentas de análise estática atuam como guardiões da qualidade, rejeitando automaticamente códigos fora do padrão antes que eles cheguem à revisão humana.

4 Evidências Visuais

Abaixo apresentamos as evidências coletadas durante a auditoria no repositório oficial.

4.1 Estrutura de Automação

A Figura 2 comprova a existência de múltiplos workflows de CI definidos no GitHub Actions.

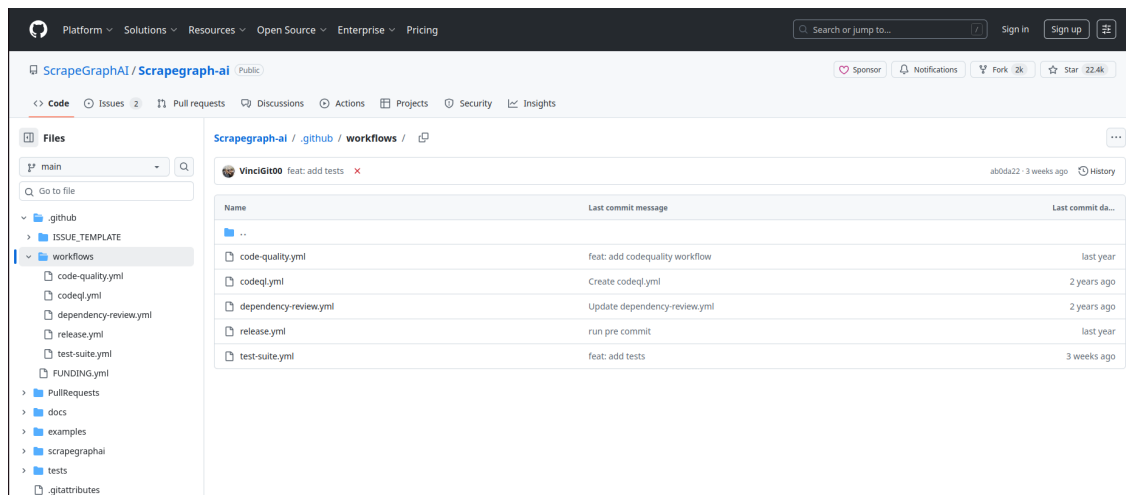


Figura 2: Diretório `.github/workflows` contendo as definições de CI/CD.

4.2 Execução e Deploy Contínuo

A Figura 3 demonstra a execução prática do pipeline. Destaca-se o comentário do bot *github-actions*, confirmando que o projeto possui **Entrega Contínua** (CD) funcional, gerando releases automaticamente após o merge.

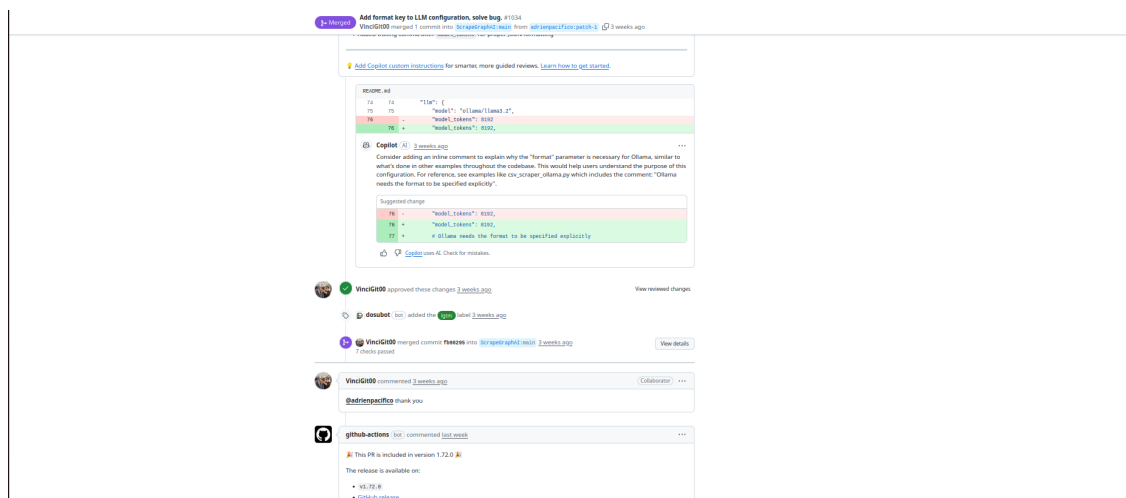


Figura 3: Pull Request fechado mostrando "7 checks passed" e a automação de release.

4.3 Configuração do Quality Gate

A Figura 4 detalha o script responsável pela análise estática, evidenciando o uso de *uv* para gerenciamento rápido de dependências e a lógica condicional do *Pylint*.

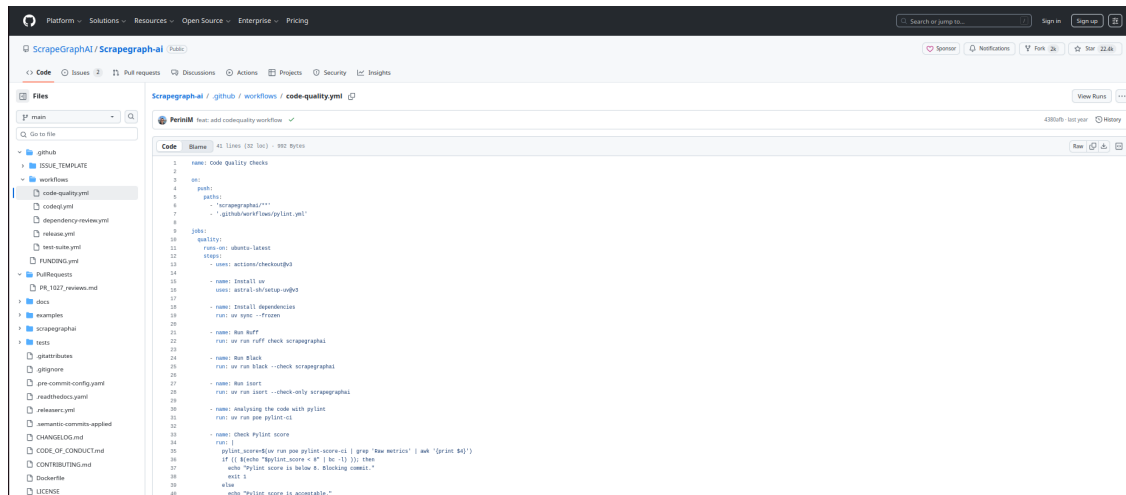


Figura 4: Trecho do arquivo code-quality.yml definindo as regras de bloqueio.

5 Conclusão do Diagnóstico

O diagnóstico conclui que o **Scrapegraph-ai** possui uma cultura de DevOps estabelecida. Diferente de projetos que sofrem com processos manuais e gargalos de deploy, este projeto utiliza automação para:

1. Garantir a integridade do código via testes e linter a cada commit.
2. Eliminar o erro humano no processo de versionamento e release.

Para a **Etapa 2 (Implementação Prática)**, o desafio da equipe será replicar este ambiente de qualidade no *Fork*, configurando um pipeline que respeite essas mesmas restrições de qualidade, garantindo que o desenvolvimento continue sustentável e seguro.